

Analisi Approfondita dei Cambiamenti: Confronto Evolutivo tra Sprint 0 e Sprint 1 nel Modello QAK

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 03, 2026

1. Introduzione e Filosofia di Progetto

Il presente documento offre una panoramica strutturata ed essenziale delle differenze e delle evoluzioni architetturali tra il modello QAK dello **Sprint 0** (`/sprint0/src/code.qak`) e il prototipo eseguibile dello **Sprint 1** (`/sprint1/prototype/codice_con_tutti_i_componenti.qak`).

L'evoluzione del codice testimonia l'applicazione pratica del concetto di “**zooming architetturale**” del corso:

- **Sprint 0 (Modello di Dominio):** Definisce il lessico, gli attori principali e i confini del sistema. È un modello di analisi non eseguibile, in cui le logiche interne degli attori sono abbozzate tramite commenti testuali.
- **Sprint 1 (Prototipo Eseguitibile del Core):** Risolve il primo sottoproblema chiave (**Separation of Concerns**), ovvero la correttezza algoritmica del ciclo di carico gestito da `cargoservice`. Per abilitare l'esecuzione immediata e il collaudo formale senza hardware reale, introduce il pattern dei **Collaboratori Simulati (Mock)**.

Nota: Obiettivo primario dello Sprint 1: Dimostrare la validità logica della Macchina a Stati Finiti dell'orchestratore, isolando l'algoritmo da problematiche esterne come latenze di rete, guasti hardware o persistenza su database.

2. 1. Evoluzione della Topologia di Rete (Contesti QAK)

Una delle differenze più evidenti tra i due modelli riguarda la gestione dei **Contesti (Context)**. La disposizione degli attori nei contesti determina se la comunicazione avviene via messaggi locali in memoria o tramite protocollo di rete TCP.

PARA-METRO	SPRINT 0 (ANALISI)	SPRINT 1 (PRO-TOTIPO)	MOTIVAZIONE IN-EGGERISTICA
Topologia	4 Contesti elementari distanti tra loro	1 Contesto Unico (<code>ctxcargoservice</code> :8050)	Focalizzazione sulla correttezza logica della FSM senza introdurre complessità di rete premature.
Meccanismo di Comunicazione	TCP/IP tra nodi di rete separati	Scambio messaggi locale (in memoria, stessa JVM)	Elimina latenza e overhead di serializzazione, rendendo il debug formale e immediato.
Preparazione per Sprint 2	N/A	Architettura multi-contesto già archiviata in <code>utils/prototype/</code>	La separazione in contesti fisici distribuirà gli attori sui nodi reali nel prossimo Sprint.

2.1. Perché un Contesto Unico nello Sprint 1?

Nello Sprint 0 si era ipotizzata una rete distribuita su 4 nodi per mappare la realtà fisica del porto navale. Tuttavia, nel prototipo formale dello **Sprint 1** tutti i 7 attori (`cargoservice` , `hold` , `sonarmock` , `markerdevice` , `ledmock` , `ioportmock` , `cargorobotmock`) sono raggruppati sotto il contesto `ctxcargoservice` .

Questa scelta risponde a un principio ingegneristico rigoroso: **isolare la verifica logica da quella di deployment**. Se avessimo distribuito subito i mock su socket TCP separati, eventuali fallimenti nei test avrebbero potuto essere causati da problemi di rete (firewall, porte occupate, serializzazione) anziché da errori nella logica di carico.

Nota: Ponte verso lo Sprint 2: I progetti archiviati in `utils/prototype/` contengono già l'architettura divisa in contesti separati (`ctxcustomer` , `ctxdevices` , `ctxrobot`). Rappresentano la base evoluta pronta per essere impiegata nello Sprint 2, quando i mock lasceranno il posto ai dispositivi fisici.

3. 2. Lessico e Protocolli: Continuità e Arricchimento

Il protocollo di interazione tra gli attori mostra sia una coerenza assoluta nelle specifiche di business, sia una necessaria espansione tecnica per rendere eseguibili i componenti simulati.

3.1. A. Continuità nel Protocollo Cliente-Servizio (100%)

Le interazioni di alto livello che governano la richiesta di carico tra il cliente (`ioport`) e l'orchestratore (`cargoservice`) non sono cambiate di una sola virgola:

```
Request load_request      : loadRequest(none)
Reply  load_accepted      : loadAccepted(SLOTID) for load_request
Reply  load_retrylater    : loadRetryLater(none) for load_request
Reply  load_refused       : loadRefused(none) for load_request
```

Questo conferma la solidità dell'Analisi dei Requisiti svolta nello Sprint 0.

3.2. B. Arricchimento dei Messaggi per la Simulazione

Per far dialogare tra loro i componenti in modo asincrono, nello Sprint 1 sono stati formalizzati nuovi messaggi non presenti nel codice di analisi:

- **Interazione con la Stiva (`hold`):** Introdotto il pattern **Request/Reply** con `get_slot` e `slot_reserved` / `hold_full` . È fondamentale avere una risposta certa prima di poter accogliere o rifiutare la richiesta del cliente. Aggiunto anche il `Dispatch free_slot` per liberare la cella in caso di annullamento.
- **Sensori e Attuatori:**
 - `Event sonardata` : Emesso dal sonar per comunicare la presenza dell'ostacolo.
 - `Dispatch set_service_status` : Per forzare il sistema in `working` o `outofservice` .
 - `Dispatch led_ctrl` : Comando unidirezionale verso il LED (`on` , `off` , `blink`).

- `Request robot_move` / `Reply robot_done` : Per comandare e attendere il completamento di ogni movimentazione del robot.
- `Request mark_container` / `Reply marking_done` : Per gestire l’etichettatura.

4. 3. Evoluzione della Macchina a Stati del `cargoservice`

L’attore orchestratore ha subito la trasformazione architetturale più profonda, passando da un modello descrittivo a una FSM pienamente reattiva.

4.1. A. Da uno stato di attesa (`work`) a `disengaged` / `engaged`

Nello Sprint 0 il sistema attendeva le richieste in un generico stato `work` . Nello Sprint 1, per aderire fedelmente alla terminologia esplicita del committente (“**considera il sistema come engaged... altrimenti disengaged**”), lo stato `work` è stato eliminato e sdoppiato:

- `disengaged` : Sistema libero, LED spento, pronto a ricevere un nuovo container.
- `engaged` : Sistema impegnato nella gestione di un carico, LED lampeggiante.

4.2. B. Guardia Booleana sulle Precondizioni

Le precondizioni di accettazione, precedentemente descritte solo a parole, sono state modellate con una guardia formale in stile Macchina di Moore:

```
if [# CargoState == "engaged" || !ServiceWorking || IOPortOccupied #] {
    replyTo load_request with load_retrylater : loadRetryLater(none)
} else {
    request hold -m get_slot : getSlot(none)
}
```

4.3. C. Srotolamento Asincrono (“Unrolling”) del Ciclo di Carico

Nello Sprint 0 l’intero processo del robot (spostamento in stiva, etichettatura, deposito finale) era riassunto in un singolo blocco di commenti dentro lo stato di accettazione. Nello Sprint 1, per rispettare il paradigma ad attori non-bloccante, questo flusso è stato “srotolato” in **7 stati sequenziali e reattivi**:

1. `wait_for_slot` : Attende la risposta asincrona dalla `hold` .
2. `accept_request` / `refuse_request` : Invia la risposta finale alla `ioport` .
3. `handle_sonar` : Rileva il posizionamento fisico del container sulla pedana.
4. `do_robot_job` : Comanda al robot lo spostamento verso `slot5` .
5. `mark_container` : Richiede l’intervento del `markerdevice` .
6. `move_to_reserved_slot` : Comanda al robot il deposito nello slot riservato.
7. `finish_job` : Spegne il LED, reimposta il sistema `disengaged` e conclude il ciclo.

5. 4. Confronto Dettagliato degli Stati (`cargoservice`)

Per facilitare la lettura, la tabella seguente confronta puntualmente la struttura interna dell'orchestratore nei due Sprint, evidenziando le motivazioni di ogni transizione.

STATO QAK	SPRINT 0	SPRINT 1	SIGNIFICATO E MODIFICA ARCHITETTURALE
<code>s0</code>	Presente	Presente	Inizializzazione variabili di stato e connessioni.
<code>work</code>	Presente (Attesa)	Eliminato	Sostituito per chiarezza terminologica da due stati distinti.
<code>disengaged</code>	Assente	Nuovo	Sistema libero. Il LED è spento e si attendono richieste.
<code>engaged</code>	Assente	Nuovo	Sistema occupato. Il LED lampeggia, preconditioni attive.
<code>handle_load_request</code>	Solo commenti	Logica Es-eguibile	Valida la guardia booleana e interroga la stiva (<code>get_slot</code>).
<code>wait_for_slot</code>	Assente	Nuovo	Attesa asincrona e non-bloccante della risposta dall'hold.
<code>accept_request</code> / <code>refuse_request</code>	Assenti (inline)	Nuovi	Smistamento formale delle risposte al cliente (<code>ioport</code>).
<code>handle_sonar</code> / <code>update_service</code>	Assenti	Nuovi	Gestione reattiva degli eventi di presenza e guasto sonar.
<code>do_robot_job</code>	Solo commento	Nuovo	Richiesta al robot di prelievo e trasporto verso <code>slot5</code> .
<code>mark_container</code>	Solo commento	Nuovo	Attesa completamento etichettatura da parte del marker.
<code>move_to_reserved_slot</code>	Solo commento	Nuovo	Richiesta di deposito finale nello slot riservato dall'hold.
<code>finish_job</code>	Assente	Nuovo	Chiusura transazione, spegnimento LED, ritorno a <code>disengaged</code> .
<code>handle_deposit_timeout</code>	Assente	Nuovo	Scadenza di 30s. Libera lo slot in stiva (<code>free_slot</code>).

6. 5. Impatto sulla Testabilità e Automazione (JUnit)

La ristrutturazione effettuata nello Sprint 1 trasforma il progetto da uno studio teorico a un sistema verificabile con strumenti industriali:

1. **Esecuzione e Debug:** Il prototipo genera attori Kotlin reali. L'esecuzione nel contesto unico `ctxcargoservice` garantisce un collaudo deterministico e veloce di tutti i rami della FSM.
2. **Resilienza Rilevabile:** I casi limite e gli errori ambientali sono ora testabili. Il guasto del sonar o il ritardo nel deposito del container attivano transizioni di timeout reali e verificabili.
3. **Suite di Collaudo JUnit:** I test plan teorici (T1, T2, T3) sono stati tradotti nella classe di test automatizzata `TestCargoServiceCore.kt`. I test agiscono come client che si connettono al server QAK, inviano richieste Prolog (`msg(load_request, ...)`) e asseriscono le risposte ricevute in modo ripetibile e rigoroso.